Smart Contract Security Analysis Report

Generated on: 2026-04-22 01:20:40

1. Executive Summary

This report provides a consolidated view of the security analysis performed on 15 smart contract(s). In total, 15 potential vulnerabilities were identified across 10 contract(s).

2. Findings Overview

Contract Name	Total Issues	Status
AccessControl.sol	1	VULNERABLE
DelegatecallUnsafe.sol	1	VULNERABLE
FrontRunning.sol	3	VULNERABLE
Reentrancy.sol	1	VULNERABLE
SelfDestruct.sol	1	VULNERABLE
Shadowing.sol	0	SECURE
TimestampDependence.sol	4	VULNERABLE
TxOrigin.sol	1	VULNERABLE
UncheckedCalls.sol	0	SECURE
contract_1.sol	1	VULNERABLE
contract_2.sol	0	SECURE
contract_3.sol	0	SECURE
contract_4.sol	1	VULNERABLE
contract_5.sol	0	SECURE
contract_7.sol	1	VULNERABLE

Smart Contract Security Analysis Report

Generated on: 2026-04-22 01:20:40

3. Detailed Vulnerability Analyses

Contract: AccessControl.sol

Static Analysis Findings (Slither)

[CRITICAL] Missing Access Control

Function: changeOwner | Line: 13

Explanation:

Function 'changeOwner' modifies a privileged variable at line 13 but lacks an access control modifier (e.g., onlyOwner). Any external caller can exploit this to seize control of the contract.

Suggest

Add an 'onlyOwner', 'onlyAdmin', or equivalent modifier to restrict access to authorized users.

Contract: contract_1.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 13

Explanation:

Function 'withdraw' makes an external call at line 13 before updating state variables. This violates the Checks-Effects-Interactions pattern and may allow an attacker to re-enter the contract.

Suggest

Update all state variables before making external calls (Checks-Effects-Interactions) or add a ReentrancyGuard (e.g., nonReentrant modifier).

Contract: contract_4.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 11

Explanation:

Function 'withdraw' makes an external call at line 11 before updating state variables. This violates the Checks-Effects-Interactions pattern and may allow an attacker to re-enter the contract.

Suggest

Update all state variables before making external calls (Checks-Effects-Interactions) or add a ReentrancyGuard (e.g., nonReentrant modifier).

Smart Contract Security Analysis Report

Generated on: 2026-04-22 01:20:40

Contract: contract_7.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 10

Explanation:

Function 'withdraw' makes an external call at line 10 before updating state variables. This violates the Checks-Effects-Interactions pattern and may allow an attacker to re-enter the contract.

Suggest

Update all state variables before making external calls (Checks-Effects-Interactions) or add a ReentrancyGuard (e.g., nonReentrant modifier).

Contract: DelegatecallUnsafe.sol

Static Analysis Findings (Slither)

[CRITICAL] Dangerous Delegatecall

Function: executeWith | Line: 38

Explanation:

Function 'executeWith' performs a delegatecall to a non-constant address ('_target') at line 38. delegatecall executes foreign bytecode inside this contract's own storage context. If the destination is user-controlled or mutable, an attacker can corrupt storage, steal ownership, or drain funds.

Suggest

Ensure the delegatecall target is a 'constant' or 'immutable' address set only in the constructor. Never delegatecall to an address passed as a parameter or stored in a mutable state variable. Prefer OpenZeppelin's upgradeable proxy patterns which enforce these constraints.

Contract: FrontRunning.sol

Static Analysis Findings (Slither)

[MEDIUM] Front-Running: ERC-20 Approve Race

Function: approve | Line: 22

Explanation:

Function 'approve' sets an allowance directly. An approved spender watching the mempool can call transferFrom() with the OLD allowance before this update is mined, then call it again with the NEW allowance ? spending both the old and new amounts.

Suggest

Replace direct approve() with increaseAllowance() / decreaseAllowance() (OpenZeppelin ERC-20 extension). Alternatively, require the caller to first reset the allowance to zero: require(allowance[msg.sender][spender] == 0 || amount == 0).

Smart Contract Security Analysis Report

Generated on: 2026-04-22 01:20:40

[MEDIUM] Front-Running: Timestamp-Dependent Payable

Function: flashSale | Line: 49

Explanation:

Payable function 'flashSale' uses block.timestamp to determine the outcome (e.g., auction deadline, lottery window). A miner or MEV bot can observe this transaction and manipulate the timestamp by ~15 seconds to decide who wins the time-sensitive competition.

Suggest

Use a commit-reveal scheme so transaction intent is hidden until after the block is committed. For auctions, consider a sealed-bid / blind-auction pattern. Avoid using block.timestamp as the sole arbiter of financial outcomes.

[MEDIUM] Timestamp Dependence

Function: flashSale | Line: 50

Explanation:

Function 'flashSale' uses 'block.timestamp' in conditional logic at line 50. Miners/validators can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auction deadlines, or random-seed generation.

Suggest

Avoid relying on block.timestamp for exact timing or randomness. For timelocks, use sufficiently large windows (> 15 minutes) so miner manipulation is economically irrelevant. For randomness, use a commit-reveal scheme or a verifiable random function (VRF) such as Chainlink VRF.

Contract: Reentrancy.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 16

Explanation:

Function 'withdraw' makes an external call at line 16 before updating state variables. This violates the Checks-Effects-Interactions pattern and may allow an attacker to re-enter the contract.

Suggest

Update all state variables before making external calls (Checks-Effects-Interactions) or add a ReentrancyGuard (e.g., nonReentrant modifier).

Contract: SelfDestruct.sol

Static Analysis Findings (Slither)

[CRITICAL] Unprotected Self-Destruct

Function: kill | Line: 13

Explanation:

Function 'kill' contains a selfdestruct call at line 13 and has no access-control guard. Any external address can call this function, permanently destroying the contract and forwarding its entire ETH balance to an arbitrary recipient.

Smart Contract Security Analysis Report

Generated on: 2026-04-22 01:20:40

Suggest

Add an 'onlyOwner' modifier (or equivalent) to 'kill', or replace the selfdestruct pattern with a pausable/upgradeable design. If self-destruction is truly required, protect it with: require(msg.sender == owner, "Not owner").

Contract: TimestampDependence.sol

Static Analysis Findings (Slither)

[MEDIUM] Front-Running: Timestamp-Dependent Payable

Function: bid | Line: 18

Explanation:

Payable function 'bid' uses block.timestamp to determine the outcome (e.g., auction deadline, lottery window). A miner or MEV bot can observe this transaction and manipulate the timestamp by ~15 seconds to decide who wins the time-sensitive competition.

Suggest

Use a commit-reveal scheme so transaction intent is hidden until after the block is committed. For auctions, consider a sealed-bid / blind-auction pattern. Avoid using block.timestamp as the sole arbiter of financial outcomes.

[HIGH] Reentrancy (CEI Violation)

Function: bid | Line: 24

Explanation:

Function 'bid' makes an external call at line 24 before updating state variables. This violates the Checks-Effects-Interactions pattern and may allow an attacker to re-enter the contract.

Suggest

Update all state variables before making external calls (Checks-Effects-Interactions) or add a ReentrancyGuard (e.g., nonReentrant modifier).

[MEDIUM] Timestamp Dependence

Function: bid | Line: 20

Explanation:

Function 'bid' uses 'block.timestamp' in conditional logic at line 20. Miners/validators can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auction deadlines, or random-seed generation.

Suggest

Avoid relying on block.timestamp for exact timing or randomness. For timelocks, use sufficiently large windows (> 15 minutes) so miner manipulation is economically irrelevant. For randomness, use a commit-reveal scheme or a verifiable random function (VRF) such as Chainlink VRF.

[MEDIUM] Timestamp Dependence

Function: claimPrize | Line: 32

Explanation:

Function 'claimPrize' uses 'block.timestamp' in conditional logic at line 32. Miners/validators can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auction deadlines, or random-seed generation.

Suggest

Avoid relying on block.timestamp for exact timing or randomness. For timelocks, use sufficiently large windows (> 15

minutes) so miner manipulation is economically irrelevant. For randomness, use a commit-reveal scheme or a verifiable random function (VRF) such as Chainlink VRF.

Contract: TxOrigin.sol

Static Analysis Findings (Slither)

[HIGH] Vulnerable Use of tx.origin

Function: withdrawAll | Line: 13

Explanation:

Function 'withdrawAll' uses 'tx.origin' for authorization at line 13. A malicious contract can call this function on behalf of a legitimate user ? 'tx.origin' will still be the user's address, bypassing the check and giving the attacker full access.

Suggest

Replace 'tx.origin' with 'msg.sender' in function 'withdrawAll'. 'msg.sender' always refers to the immediate caller and cannot be spoofed by an intermediate contract.